Trabalho I - Máquina de Estados

1. **Visão do Cenário:**

Em nosso cenário, temos duas pessoas trabalhando juntas na criação de um jogo, um programador e um artista.
O programador é responsável por estruturar o código e implementar os assets, teoricamente, fazendo o jogo funcionar. Já o artista tem o papel de apenas criar os assets e passar para o programador.

2. **Visão de cada Agente:**

Visão do Programador:

O programador deve implementar as artes e criar o código, portanto ele tem o seguinte processo: prototipar, e em sequência programar de fato, que também inclui a implementação de assets. Ao fazer isso ele gera bugs, que devem ser corrigidos posteriormente. O programador só consegue passar para a etapa de programação assim que tem um mínimo de prototipação feita e um número necessário de assets.

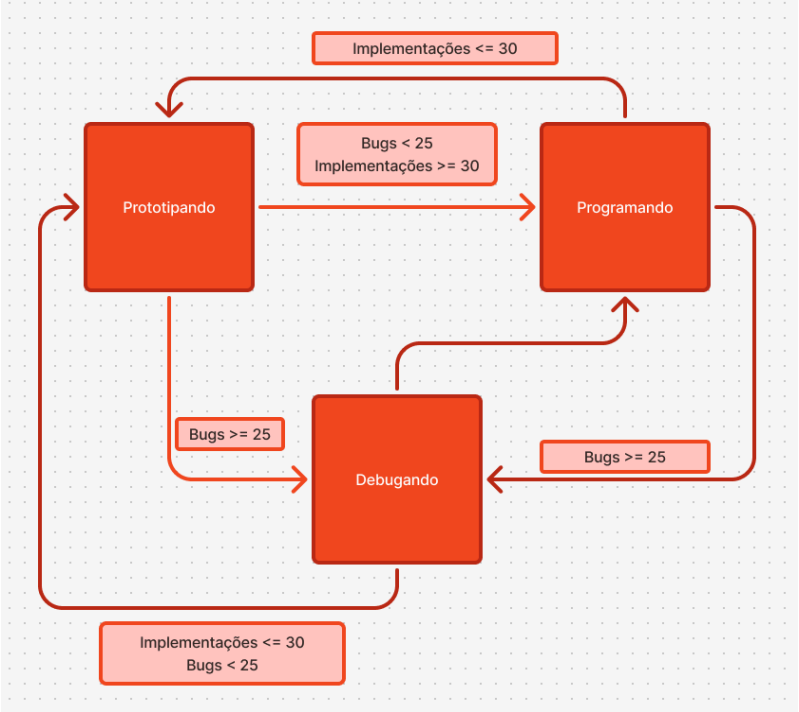
Visão do Artista:

O artista tem o papel de apenas produzir os assets, que neste caso são 2D. Ele começa pesquisando referências, e ao ter um certo nível de referências, ele começa a desenhar, ou seja, produzir assets. Quando o artista tem um número específico de assets, ele os encaminha para o programador e volta a pesquisar referências.

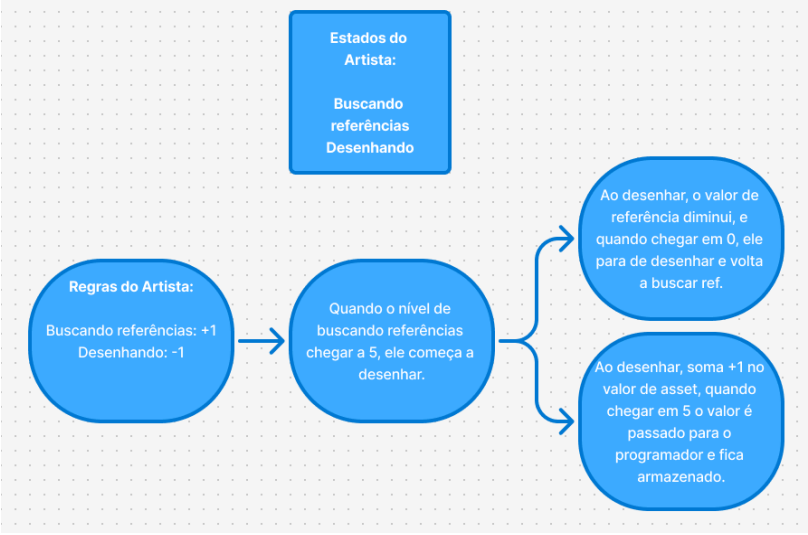
3. **Diagrama de Estados:**

Programador:




Implementações <= 30
Prototipando
Bugs < 25
Implementações >= 30
Programando
Bugs >= 25
Debugando
Bugs >= 25
Implementações <= 30
Bugs < 25

Artista:


Estados do Artista:
Buscando referências
Desenhando
Regras do Artista:
Buscando referências: +1
Desenhando: -1
Quando o nível de buscando referências chegar a 5, ele começa a desenhar.
Ao desenhar, o valor de referência diminui, e quando chegar em 0, ele para de desenhar e volta a buscar ref.
Ao desenhar, soma +1 no valor de asset, quando chegar em 5 o valor é passado para o programador e fica armazenado.

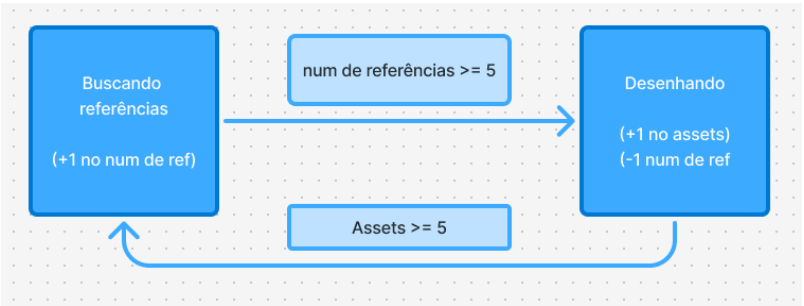
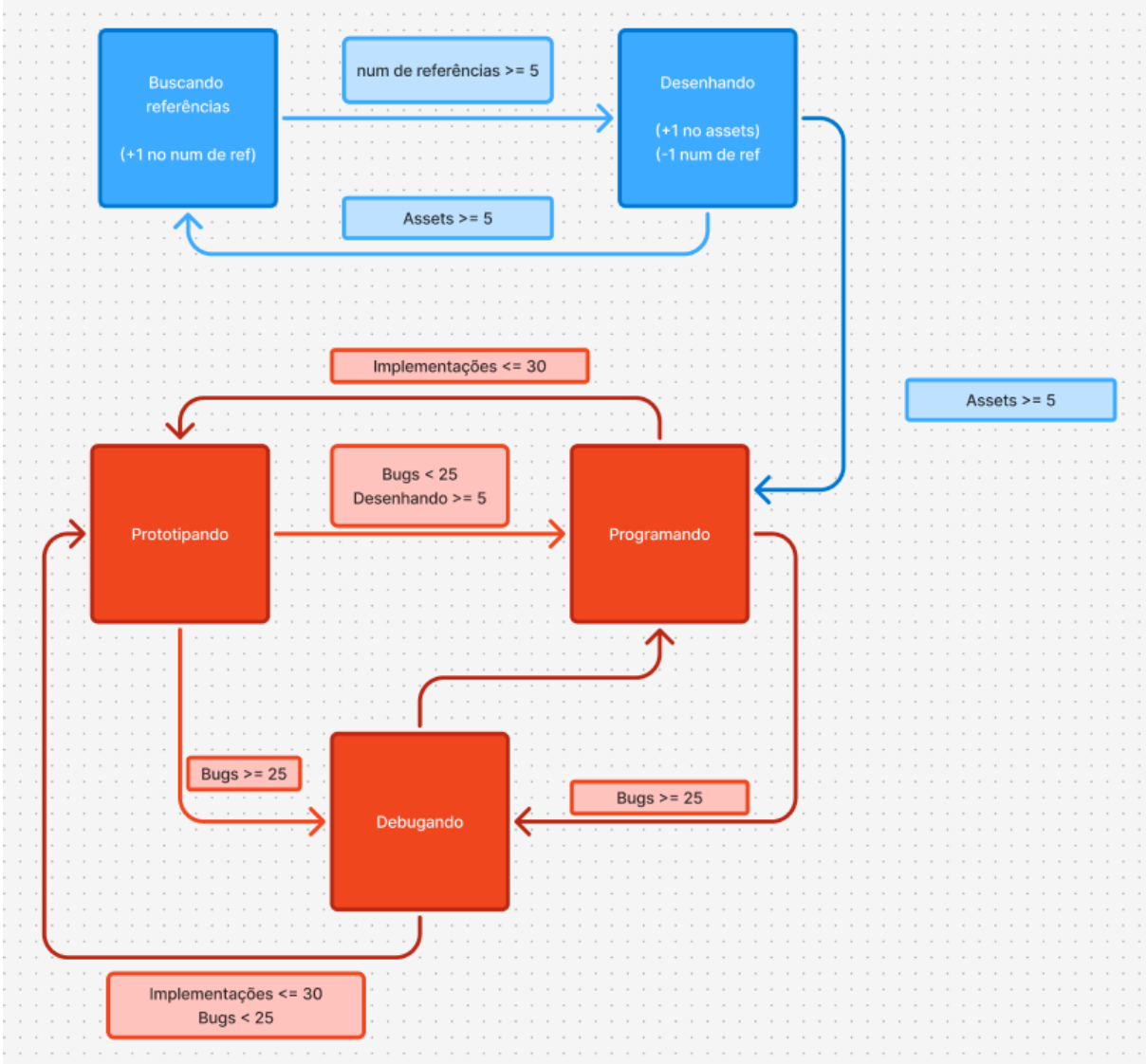

Buscando referências
(+1 no num de ref)
num de referências >= 5
Desenhando
(+1 no assets)
(-1 num de ref
Assets >= 5

Diagrama Completo:



4. **Tabela de Regras:**

Programador:

Prototipando	+ 5 implementações + 3 debugs
Programando	+ 5 bugs - 3 implementações - 1 asset
Debugando	- 5 implementações - 10 bugs

Artista:

Buscando referências	+ 1 inspiração
Desenhando	+ 1 asset - 1 inspiração

5. **Descrição das Variáveis:**

Programando - Classe Prog:

- **implementacoes**
  - Tipo: int
  - Tamanho: 32 bits
  - Valor inicial: 0
  - Finalidade: Armazena a quantidade de implementações realizadas pelo programador.
  - Alterações: aumenta em 5 durante a prototipação, diminui em 3 durante a programação e diminui em 5 durante a depuração.
  - Limiar: valor menor ou igual a 30 pode provocar a saída dos estados ProgramandoState ou DebugandoState, dependendo das condições específicas de cada estado.
- **bugs**
  - Tipo: int
  - Tamanho: 32 bits
  - Valor inicial: 0
  - Finalidade: Armazena a quantidade de bugs acumulados pelo programador.

- Alterações: aumenta em 3 durante a prototipação, aumenta em 5 durante a programação e diminui em 10 durante a depuração.
- Restrição: o valor não pode ficar negativo; durante a depuração, caso seja menor que zero, é ajustado para zero.
- Limiar: valor maior ou igual a 25 provoca a transição para o estado DebugandoState a partir dos estados PrototipandoState e ProgramandoState.

Artista - Classe Arts:

- **references**
  - Tipo: int
  - Tamanho: 32 bits
  - Valor inicial: 0
  - Finalidade: Armazena a quantidade de referências reunidas pelo artista.
  - Alterações: aumenta em 1 durante a busca por referências e diminui em 1 durante a produção de um asset.
  - Limiar: valor maior ou igual a 5 provoca a transição de InspirationState para DesenhandoState. Quando o valor chega a zero ou menos, o artista retorna à busca de referências.
- **assets**
  - Tipo: int
  - Tamanho: 32 bits
  - Valor inicial: 0
  - Finalidade: Armazena a quantidade de assets produzidos pelo artista e a quantidade disponibilizada para o programador.
  - Alterações: aumenta em 1 durante o desenho e diminui em 1 quando o programador começa a programar, ou executa uma ação no estado ProgramandoState.
  - Limiar: valor >= 5 permite que o artista retorne à busca de referências e pode permitir a transição do programador para o estado ProgramandoState, desde que as outras condições sejam atendidas.

Obs: os atributos references e assets são privados e possuem métodos de acesso e alteração definidos na classe Arts. Da mesma forma, implementações e bugs são privados e possuem métodos de acesso e alteração definidos na classe Prog.

6. Estrutura do código:

Classes principais:

Main

- Responsável por rodar a simulação.
- Instancia os objetos Arts e Prog.
- Utiliza a variável contador para controlar a quantidade de turnos da simulação, encerrando o laço após 20 turnos.
- A cada turno, atualiza primeiro o artista e depois o programador (mesmo tendo a impressão de ser ao mesmo tempo).

Prog

- Representa o programador e armazena os valores de implementações e bugs.
- Possui métodos para executar as ações de prototipação, programação e depuração.
- Controla a mudança de estados por meio do método setState().
- O método update() executa o comportamento do estado atual.

Arts

- Representa o artista e armazena os valores de referências e assets.
- Possui métodos para buscar referências e produzir assets.
- Controla a mudança de estados por meio do método setState().
- O método update() executa o comportamento do estado atual.

Classes e interface da máquina de estados:

State<C>

- Interface que define os métodos comuns aos estados.
- Possui os métodos getCharacter(), enter(), execute() e leave().
- O parâmetro genérico C representa o tipo do objeto relacionado ao estado.

AbstractState<C>

- Classe abstrata que implementa a interface State<C>.
- Armazena uma referência ao objeto associado ao estado por meio do atributo character.
- O método getCharacter() retorna o objeto armazenado.
- Os métodos enter() e leave() possuem implementações vazias, podendo ser sobrescritos pelas subclasses.
- O método execute() permanece definido pelas classes concretas que herdam essa classe.

Classes dos estados do programador:

PrototipandoState

- Aumenta em 5 o valor da variável implementações e em 3 o valor da variável bugs, por meio do método prototipando() da classe Prog.
- Se a quantidade de bugs for maior ou igual a 25, o programador passa para DebugandoState.
- Caso contrário, se a quantidade de bugs for menor que 25 e o artista possuir pelo menos 5 assets, um asset é consumido e o programador passa para ProgramandoState.

ProgramandoState

- Aumenta em 5 a variável bugs e diminui em 3 a variável implementacoes, por meio do método programando() da classe Prog.
- Consome 1 asset por execução, caso exista pelo menos um asset disponível.
- Se a quantidade de bugs for maior ou igual a 25, passa para DebugandoState.
- Caso contrário, se a quantidade de implementações for menor ou igual a 30, retorna para PrototipandoState.

DebugandoState

- Diminui em 5 a variável implementacoes e em 10 a variável bugs, por meio do método debugando() da classe Prog.
- Impede que a quantidade de bugs fique negativa, ajustando o valor para zero quando necessário.
- Quando a quantidade de implementações for menor ou igual a 30 e a quantidade de bugs for menor que 25, retorna para PrototipandoState.

Classes dos estados do artista:

InspirationState

- Aumenta em 1 a variável references, por meio do método inspiracao() da classe Arts.
- Quando a quantidade de referências for maior ou igual a 5, o artista passa para DesenhandoState.

DesenhandoState

- Aumenta em 1 a variável assets e diminui em 1 a variável referências, por meio do método desenhando() da classe Arts.

- Quando a quantidade de assets for >= 5, ou quando a quantidade de referências for <= 0, o artista retorna para InspirationState.

Funcionamento geral

- Os objetos Prog e Arts mantêm seus próprios estados e valores.
- Cada estado executa suas ações e verifica as condições para realizar transições.
- O método setState() chama leave() no estado anterior, atribui o novo estado e chama enter() no novo estado.
- A execução da simulação ocorre por meio dos métodos update() das classes Prog e Arts.

7. Resultados esperados:

Ambos os agentes trabalham separadamente até que os dois possam passar para o "programando".
O programador prototipa, corrige debugs, prototipa novamente e finalmente pode passar para programando, já que ele tem implementações e assets suficientes e poucos debugs.
Enquanto isso o artista continua pesquisando e desenhando, e passando os assets, que ficam armazenados, esperando o programador ter implementações o bastante e poucos debugs.

8. Limitações:

Acreditamos que não tenha nada que faltou ser implementado para o programa funcionar.
Provavelmente o que faria o programa ficar melhor e mais completo, seria o balanceamento dos valores e a inclusão de outros agentes com outras funções, tornando o processo de criação do jogo mais real.

9. Referências:

Para a ideia do projeto, utilizamos apenas nossas experiências como estudantes de jogos digitais. Já para o trabalho, utilizamos a referência da própria professora e ajuda externa de IA (Gemini e ChatGPT).